Appendix G

Java Logging API Overview

The Java Logging API is a supplementary topic, and *not* an exam objective. However, there could be some exam questions that utilize the Logging API to facilitate non-blocking parallel processing, such as in parallel streams. Also, an understanding of the Logging API principals and capabilities is an important practical programming skill.

G.1 Purpose of the Logging API

The purpose of the Java Logging API is to provide a simple yet flexible mechanism for writing logs to different destinations, while minimizing the effects of writing these logs on the overall performance of a Java application. A program can write messages to the standard output or standard error using `System.out` or `System.err`, respectively. However, this approach is not particularly flexible, because such write operations block the current thread. Thus they adversely affect program performance, and are totally unsuitable in any concurrent or parallel execution of code—for example, in parallel stream processing. Using standard out and standard error is also not very flexible because they do not allow a programmer to reconfigure the target destination for the log messages, nor do they provide a flexible dynamic mechanism to filter or format log messages.

The Logging API addresses the following issues:

- Provides a *configuration properties file* to configure:
  - *Log handlers*, which write log messages to different destinations
  - *Log message format* for each destination
  - *Logging levels*, which control how detailed logging should be done
- Minimizes performance implications with:
  - *Non-blocking* log writing operations
  - *Guarded logging*, which avoids unnecessary message formatting when the logger level is set in a way that a particular message is not going to reach the actual log destination

G.2 Configuring Logging

By default, the file `logging.properties` is used to configure Java logging. It is located in the `JAVA_HOME/conf` folder. (For Java version 8 or earlier, this was `JAVA_HOME/jre/lib` folder.) A different configuration file can be specified using the `java.util.logging.config.file` system property. For example:

Click here to view code image

```
java -Djava.util.logging.config.file=another-config
```

Log handlers are classes that are responsible for writing log messages to different destinations. The package `java.util.logging` provides a number of handler classes. A *logger configuration file* can be used to enable one or more handlers specified as a comma-separated list. By default, the logger configuration only sets up the *console handler*, but other handlers can be enabled as required. For example:

Click here to view code image

```
handlers = java.util.logging.FileHandler, java.util.logging.ConsoleHandler
```

Each handler has its own specific configurable properties. For example, a file handler allows setting up a naming pattern for log file names, minimum size of the file, log rollover to the next file, and so on. For example:

Click here to view code image

```
java.util.logging.FileHandler.pattern = %h/java%u.log
java.util.logging.FileHandler.limit   = 50000
java.util.logging.FileHandler.count   = 1
```

Obviously, such properties are not applicable to other types of handlers, such as the console handler. However, all handlers provided by the Java Logging API allow a logger format to be set; for example, as a simple plain text or xml format.

Click here to view code image

```
java.util.logging.ConsoleHandler.formatter = java.util.logging.SimpleFormatter
```

```
java.util.logging.FileHandler.formatter    = java.util.logging.XMLFormatter
```

It is possible to implement customized handlers by extending the `java.util.logging.Handler` class or one of its subclasses: `MemoryHandler` or `StreamHandler`. Also, it is possible to implement a customized formatter by extending the `java.util.logging.Formatter` class.

Level configuration is used to control the number of messages that are actually written to a log destination. *Logging levels* are defined by the `java.util.logging.Level` class using the following constants:

> `SEVERE` *(highest value)*
>
> `WARNING`
>
> `INFO`
>
> `CONFIG`
>
> `FINE`
>
> `FINER`
>
> `FINEST` *(lowest value)*

In addition, there is a level `OFF` that can be used to turn off logging, and a level `ALL` that can be used to enable logging of all messages.

When logging is set to a certain logging level, any log message that is associated with this logging level and levels above it will be written to the log destination. However, messages that are associated with any logging levels below the one that was set in the configuration will essentially be ignored by the logger.

For example, when the log level is set to `INFO`, the `INFO`, `SEVERE`, and `WARNING` level messages will be written to the log, but the `CONFIG`, `FINE`, `FINER`, and `FINEST` level messages will be ignored by the logger.

It is possible to implement additional logging levels by subclassing the `java.util.logging.Level` class.

Log level configuration defines a *global log level property*, but also allows overriding the level for specific loggers. For example:

```
.level=INFO
org.foo.level=SEVERE
```

It is important to note that by convention, logger names are essentially Java packages and class names. So this example implies that everywhere in the program, the global logger level is `INFO`, except for the package `org.foo`, where it is set to `SEVERE`.

It is also possible to set the logging level for a specific handler. For example:

Click here to view code image

```
java.util.logging.ConsoleHandler.level = WARNING
```

Suppose that logging has been configured to use a file logger and the console logger, and the console logger is restricted to the `WARNING` level, but no such restriction is imposed on the file logger. Also, assume that the global log level is set to `INFO`. This would mean that log messages associated with a level below `INFO` would all be ignored; only `WARNING` and `SEVERE` level message would be written to the console logger; and `INFO`, `WARNING`, and `SEVERE` level messages would be written to the file logger.

G.3 Writing Log Messages

To set up logging for a program requires a few basic steps. First, a logger object must be initialized, using the `java.util.logging.Logger.getLogger(String name)` method. By convention, the name of a logger object should correspond to the fully qualified class name from which the log is written. This way, it will match the logger naming convention used by the logger configuration file. For example:

Click here to view code image

```
package org.foo;
import java.util.logging.*;
public class Test {
  private static Logger logger = Logger.getLogger(org.foo.Test.class.getName());
  public static void main(String[] args) {
```

```
    ...
   }
  }
```

Notice that the logger name could theoretically be any string. However, using package and class names really helps to organize logger configuration. In this case, a logger config file can associate the `org.foo` logger name with a particular level of logging, which is going to affect log messages written from the `org.foo.Test` class.

Once the logger object is initialized, it can be used to write messages, using the `logger.log()` method. Each message is associated with a specific logger level and may contain any text. The `log()` method is overloaded to allow the program to pass an exception object as an additional argument so that exception details can be passed to the logger as part of the message. For example:

Click here to view code image

```
package org.foo;
import java.util.logging.*;
public class Test {
 private static Logger logger = Logger.getLogger(org.foo.Test.class.getName());
 public static void main(String[] args) {
  try {
   // any actions that may potentially throw an exception
  }catch(Exception e) {
   logger.log(Level.ERROR, "Some error message", e);
  }
  logger.log(Level.INFO, "Some other message");
 }
}
```

The `Logger` class provides a number of additional convenience methods that are equivalent to the `log()` method:

`logp()`

The *log precise* method takes additional parameters that specify a class and a method name.

`logrb()`

The *log with resource bundle* method takes a resource bundle name that may be used to localize log messages.

`entering(), exiting(), throwing()`

These are convenience methods used to log method entry, method exit, and any exception thrown at the `FINER` level.

`severe(), warning(), config(), info(), fine(), finer(), finest()`

These are convenience methods, each hardcoded to use a specific log level, rather than indicating the log level as an argument. In other words, `logger.info("My message")` is equivalent to `logger.log(Level.INFO, "My message")`.

All of these log methods are *non-blocking*. This means that when any of these log methods are invoked in the program, the current thread is not blocked, as the method call returns immediately. The logger accumulates messages produced by the different program threads, and writes these messages to the actual destinations in another thread of its own. As a result, the log messages can reach their destinations with a minimum of delay, but the performance impact on the overall program execution is minimized. This allows loggers to be utilized in concurrent and parallel execution scenarios.

G.4 Applying Guarded Logging

Finally, it is a good idea not to format log messages ahead of time. This is because it is not known whether the message would or would not be written into a log because of the possible changes to the logger configuration. This means that code in the following example is not advisable:

Click here to view code image

```
logger.log(Level.FINE, "This is " + someVar + " value");
```

In the preceding code, message concatenation is going to be performed even if the log level could be set in a way that would ignore this message. In order to prevent such unnecessary processing, it is advised to use *guarded logging*. Basically, the aim is not to do message formatting when the message is going to be discarded because of the log level settings. This could be achieved with an `if` statement or by using *message value substitution*. For example, the following `if` statement:

Click here to view code image

```
if(logger.isLoggable(Level.FINER)) {                    // (1)
  logger.log(Level.FINE, "This is " + someVar + " value");
}
```

can be witten using message value substitution:

Click here to view code image

```
logger.log(Level.FINE, "This is {0} value", someVar); // (2)
```

Notice that the code at (2) looks much cleaner than the code at (1) and is thus probably the best way to handle log messages.

G.5 Summary

The Logging API provides a simple, yet flexible non-blocking mechanism to write information about program execution, including general information as well as error messages. The certification exam does not ask any direct questions about logging, but use of loggers may be encountered in the context of other questions, specifically where non-blocking output is important, such as in the context of the Concurrency API or execution of parallel streams.

Previous chapter
‹ Appendix F. Annotated Answers to Mock Exam

Next chapter
Part II: Index ›